Local Consistency, Constraint Propagation & Global Constraints

Overview and Context

Constraint Programming (CP) differentiates itself from pure brute-force search by utilizing a powerful, active inference engine. Rather than blindly guessing variable assignments until a solution is found or a failure occurs, a CP solver applies **Constraint Propagation**. This is the mechanism of proactively examining constraints to detect and remove incompatible values—values that cannot possibly be part of any valid solution—from the domains of unexplored variables.

This lesson explores the theoretical foundations and operational mechanics of this inference engine. The primary learning objectives are to master the mathematical definitions of **Local Consistency** (the precise state a constraint reaches after successful propagation), to understand how interacting algorithms converge to a global "fixed-point", and to leverage **Global Constraints**. These advanced constraints use dedicated algorithms to enforce strict consistencies across complex, recurring combinatorial substructures, drastically reducing the search space.

1. Local Consistency: GAC vs. Bounds Consistency

Explanation: To mathematically guarantee that a constraint has been fully propagated, we define specific properties it must satisfy, known as local consistencies. It is termed "local" because it evaluates one constraint at a time. The solver's objective is to achieve these consistencies by removing unsupported values. Depending on the computational cost and the nature of the domain, solvers typically aim for either **Generalised Arc Consistency (GAC)**, which rigorously checks every individual value, or **Bounds Consistency (BC)**, a relaxed, cheaper consistency ideal for totally ordered domains (like integers) where only the range limits are evaluated.

- **Support:** A tuple $(d_1, \ldots, d_k)$ that belongs to the set of allowed combinations of a constraint $C \subseteq D(X_1) \times \cdots \times D(X_k)$.
- **Generalised Arc Consistency (GAC):** A constraint $C(X_1, \ldots, X_k)$ is GAC if and only if: for all variables $X_i \in \{X_1, \ldots, X_k\}$, and for all values $v \in D(X_i)$, the value v belongs to a valid support. When $k = 2$, this is simply called **Arc Consistency (AC)**.
- **Bounds Consistency (BC):** Relaxes the domain $D(X_i)$ into a continuous interval $[min(X_i) \ldots max(X_i)]$. A constraint is BC if and only if for all variables X_i, both $min(X_i)$ and $max(X_i)$ belong to a **bound support** (a tuple where every $d_j \in [min(X_j) \ldots max(X_j)]$).

Examples & Applications

Translating GAC vs. BC into Practice: * **AC Example:** Given $D(X_1) = \{1, 2, 3\}$, $D(X_2) = \{2, 3, 4\}$, and constraint $C : X_1 = X_2$. The value $1 \in D(X_1)$ and $4 \in D(X_2)$ have no support. Enforcing AC removes them, leaving $D(X_1) = \{2, 3\}$ and $D(X_2) = \{2, 3\}$. * **BC Weakness on non-monotonic constraints:** Consider C : alldifferent($[X_1, X_2, X_3]$) with domains $D(X_1) = D(X_2) = D(X_3) = \{1, 3\}$. * *BC Check:* $min = 1$ and $max = 3$ belong to bound supports (e.g., $(1, 2, 3)$ is a valid bound tuple since $2 \in [1..3]$), so BC prunes nothing. * *GAC Check:* GAC correctly detects a failure because 3 variables cannot be assigned unique values from a set of only 2 distinct values ($\{1, 3\}$). Thus, BC requires more search to find the inconsistency, while GAC detects it instantly.

2. Propagation Algorithms and The Fixed-Point

Explanation: In reality, a CSP consists of many intersecting constraints. A **propagation algorithm** is the operational code that achieves a specific local consistency (like GAC or BC) by removing inconsistent values. Because constraints share variables, pruning a value from $D(X_1)$ due to constraint C_A might destroy a support previously relied upon by constraint C_B. Therefore, propagation algorithms must dynamically "wake up" and re-evaluate their constraints in a continuous chain reaction. This operational loop ceases only when no further domain reductions can be made across the entire system—a state referred to as the **fixed-point**.

- **Trigger Events:** A dormant propagation algorithm wakes up when relevant changes occur:
  - When a variable's domain changes (triggers GAC algorithms).
  - When domain bounds change (triggers BC algorithms).
  - When a variable is strictly assigned to a single value.
- **Algorithm Complexity:** Generic AC propagation takes $O(d^2)$ time (where $d = |D(X_i)|$). However, specialized constraints are faster. For example, $X_1 \neq X_2$ only takes $O(1)$ time because it only needs to wake up when one variable is reduced to a single value.

Examples & Applications

The Wake-Up Loop in Action: Given $D(X_1) = D(X_2) = D(X_3) = \{1, 2, 3\}$ with C_1 : alldifferent($[X_1, X_2, X_3]$), $C_2 : X_2 < 3$, and $C_3 : X_3 < 3$. 1. Evaluate C_1: It is currently GAC; no pruning. 2. Evaluate C_2: Prunes 3 from $X_2 \implies D(X_2) = \{1, 2\}$. 3. Evaluate C_3: Prunes 3 from $X_3 \implies D(X_3) = \{1, 2\}$. 4. **The Wake-up:** The pruning of X_2 and X_3 destroys the supports for $X_1 = 1$ and $X_1 = 2$ in C_1. Therefore, C_1 *wakes up* again and re-propagates, forcing $X_1 = 3$.

3. Global Constraints and Formulas

Explanation: To model complex, real-world problems (like routing or scheduling), CP uses **Global Constraints**. These encapsulate non-binary, recurring combinatorial substructures. By recognizing the macro-structure of a problem rather than viewing it as a disconnected soup of logical operators, the solver can deploy mathematically specialized, highly efficient propagation algorithms. This closes the gap between the high-level problem statement and the low-level solving mechanics.

- **Counting Constraints:**
  - `alldifferent([X1, ..., Xk])`: Holds iff $X_i \neq X_j$ for all $i < j \in \{1, \ldots, k\}$.
  - `gcc([X1..Xk], [v1..vm], [O1..Om])`: Global Cardinality Constraint. Holds iff for all $j \in \{1..m\}$, the occurrence limit $O_j = |\{i \mid X_i = v_j, 1 \leq i \leq k\}|$.
  - `among([X1..Xk], s, l, u)`: Holds iff the number of variables taking values from set s is between l and u: $l \leq |\{i \mid X_i \in s\}| \leq u$.
- **Sequencing & Scheduling:**
  - `sequence(l, u, q, X, s)`: Ensures `among(..., s, l, u)` holds for *every* sliding window of q consecutive variables.
  - `disjunctive([S1..Sk], [D1..Dk])`: Non-overlapping tasks (capacity 1). Holds iff $\forall i < j : (S_i + D_i \leq S_j) \vee (S_j + D_j \leq S_i)$.
  - `cumulative([S1..Sk], [D1..Dk], [R1..Rk], C)`: Shared resources. Holds iff at any time u, the sum of required resources R_i for active tasks ($S_i \leq u < S_i + D_i$) does not exceed capacity C.
- **Ordering (Symmetry Breaking):**
  - `lex_leq([X1..Xk], [Y1..Yk])`: Lexicographic ordering. Holds iff $X_1 \leq Y_1 \wedge (X_1 = Y_1 \rightarrow X_2 \leq Y_2) \ldots$.

Examples & Applications

Breaking Symmetry with Lexicographic Ordering: When assigning items to symmetric bins, modelled as a matrix of Boolean variables, many assignments are structurally identical (e.g., swapping column vectors X and Y). Imposing `lex_leq(X, Y)` forces the solver to accept only the mathematically sorted permutation, mathematically eliminating the symmetric counterparts from the search tree.

4. Decomposition vs. Dedicated Algorithms

Explanation: To achieve GAC on a global constraint, solvers use two approaches. **Decomposition** breaks the global constraint into simpler, primitive constraints (e.g., decomposing `alldifferent` into a network of $X_i \neq X_j$). While easy, this severely weakens propagation because the solver loses the global view of the substructure. To achieve true GAC efficiently, solvers use **Dedicated Propagation Algorithms** that embed advanced theoretical math (like flow

theory or graph theory) directly into the inference engine.

- **Decomposition Weakness:** Decomposing `alldifferent([X1, X2, X3])` where $D(X_i) = \{1, 2\}$ results in individual constraints $X_1 \neq X_2$, $X_2 \neq X_3$, $X_1 \neq X_3$. Individually, each pair is AC, so decomposition prunes nothing, whereas true GAC immediately fails the global constraint.
- **Dedicated Algorithms (Régin's Algorithm):** Achieves true GAC on `alldifferent` in polynomial time by mapping the variables and domains into a **Bipartite Graph**.
- **Generic Tables:** If a constraint is highly irregular (like valid words in a crossword), a `table([X1..Xk], dictionary)` constraint is used, leveraging the solver's generic, highly optimized GAC table-checking algorithms.

Examples & Applications

Régin's Bipartite Graph for `alldifferent`: To maintain GAC on `alldifferent`, Régin's algorithm builds a bipartite graph with Variables on one side (U) and Values on the other (V). An edge exists if a value is in a variable's domain. A valid solution corresponds exactly to a **Maximal Matching** (the largest set of non-sharing edges covering all variables). The algorithm uses graph theory to compute all maximal matchings and automatically prunes any edge (variable-value assignment) that does not belong to at least one maximal matching, guaranteeing perfect GAC.

Synthesis and Conclusions

The true power of Constraint Programming lies in Constraint Propagation—the continuous, inferential elimination of impossible values. By mathematically defining states like Generalised Arc Consistency (GAC) and Bounds Consistency (BC), we provide rigorous targets for our propagation algorithms. As these algorithms interact dynamically, they rapidly prune the search space until reaching a fixed-point. Because complex problems yield overwhelmingly large search trees, the introduction of Global Constraints is vital. Whether through graph-based dedicated algorithms like Régin's bipartite matching or sequence bounds, Global Constraints utilize the macroscopic mathematical structure of a problem to execute domain reductions that isolated, decomposed constraints could never deduce.

Essential Glossary

- **Local Consistency:** A formal property dictating the specific conditions under which inconsistent values must be removed from domains.
- **Support:** A valid tuple of values that satisfies the mathematical definition of a constraint.
- **Generalised Arc Consistency (GAC):** A strict local consistency ensuring every single value in every domain belongs to at least one valid support tuple.

- **Bounds Consistency (BC):** A relaxed local consistency for ordered domains ensuring only the minimum and maximum domain limits belong to a valid support tuple.
- **Fixed-Point:** The operational state where all constraints have been propagated and no further domain reductions can be deduced by the algorithms.
- **Global Constraint:** A macro-constraint capturing complex combinatorial substructures (e.g., `alldifferent`, `cumulative`) enabling specialized, highly potent inference.
- **Bipartite Graph:** A graph whose vertices are divided into two disjoint sets (U, V) with edges only connecting U to V; fundamental to propagating `alldifferent`.
- **Maximal Matching:** The largest possible set of non-adjacent edges in a bipartite graph, representing a valid assignment in permutation constraints.